

Projet · Forme Comtoise

La nouvelle API des clubs : migration, cache et rapport en arrière-plan

Joseph Azar · Maître de Conférences
UMLP · Chercheur FEMTO-ST
joseph.azar@umlp.fr

R510 · Nouvelles bases de données · BUT3 S5

Contexte. Forme Comtoise est un réseau de cinq salles de sport en Franche-Comté. Chaque étudiant tient le rôle du développeur backend chargé de reprendre l'application des clubs. Le message de la responsable technique, ci-dessous, constitue le cahier des charges. Ses pièces jointes sont dans l'archive.

Archive	Contenu
projet-forme-comtoise.pdf	l'énoncé
data/forme-comtoise.sqlite	la base de l'application actuelle : 8 tables, 17 Mo
data/passages-2024-2026.csv	l'export des tournois : 1 133 982 lignes, 46 Mo

Hélène Cattin · responsable technique, Forme Comtoise

À : développeur backend

Date : jeudi 24 septembre 2026

Objet : la nouvelle API des clubs

Pièces jointes : forme-comtoise.sqlite, passages-2024-2026.csv

Bonjour,

Contexte. Cinq clubs : Besançon Centre, Besançon Valentin, Montbéliard, Dole et Pontarlier. Environ 3 700 adhérents actifs, une trentaine de coaches, 150 cours collectifs par semaine. L'application actuelle, livrée par un prestataire, repose sur la base SQLite jointe. Ses temps de réponse se dégradent le dimanche soir. La migration vers MongoDB doit être terminée avant janvier, pic des inscriptions.

1. Migration. La base compte huit tables ; la cible est de cinq collections au plus. Chaque table qui disparaît est justifiée par écrit. Les remontées des clubs ci-dessous servent de base à ces décisions.

- (a) « On a augmenté Premium l'an dernier. Les anciens abonnés gardent leur prix, c'est écrit dans leur contrat. Le nouveau prix vaut pour les contrats signés à partir du jour où on le change. »
Patrick, directeur de Besançon Centre
- (b) « Quand un adhérent arrive à l'accueil, je veux voir sa formule actuelle et toutes celles qu'il a eues avant, sur un seul écran. »
Sabrina, accueil de Pontarlier
- (c) « Un cours, c'est trente places au maximum. Cinq minutes avant, j'imprime la liste d'appel. »
Julie Perrin, coach
- (d) « Monsieur Grosjean vient depuis 2020. Il a réservé plus de 550 cours depuis que les réservations existent, et il veut pouvoir tous les retrouver. »
accueil de Besançon Centre
- (e) « Quand je suis devenue Julie Perrin, le planning a continué d'afficher Martin pendant trois semaines. »
Julie Perrin
- (f) « Un coach qui quitte le réseau n'est jamais supprimé. Ses anciens cours restent dans les statistiques, avec son nom. »
la direction
- (g) « Une activité ne change jamais de nom. Quand on change le concept, on crée une nouvelle activité. »
la direction

(h) « *La capacité d'un club est fixée par la commission de sécurité. Elle change quand on fait des travaux.* » Patrick

(i) Les spécialités des coaches ont été saisies à la main, sous des formes variées (Yoga, Pilates, yoga ,pilates, PILATES;Stretching), et doivent devenir exploitables.

Dans SQLite, les dates sont stockées sous forme de texte et les booléens sous forme de 0 et 1. Les heures de cours sont en heure de Paris.

2. API. Node.js, Express et Mongoose. Pas d'interface : les tests se font avec Postman. Fonctions attendues :

- les clubs et leurs horaires, les formules et leur prix actuel ;
- le planning d'un club pour une semaine, avec les places restantes de chaque cours ;
- réserver une place, annuler une réservation ;
- la fiche d'un adhérent : sa formule actuelle, l'historique de ses formules, ses dix dernières séances ;
- inscrire un nouvel adhérent avec sa première formule ;
- modifier le prix d'une formule, modifier un coach, annuler une séance (coach malade : la séance disparaît du planning) ;
- pour la direction : le taux de remplissage et le taux d'absence par activité sur une période, le classement des coaches, et le chiffre d'affaires mensuel d'un club. Pour ce dernier, chaque contrat en cours le 1^{er} du mois compte pour un mois plein, au prix du contrat.

Règles de réservation : une place n'est attribuée qu'une fois, y compris quand deux demandes arrivent en même temps ; la formule doit inclure les cours ; seule la formule Premium donne accès aux autres clubs.

3. Performances. Le planning est la page la plus consultée, en particulier le dimanche soir, quand les adhérents réservent leur semaine. La direction consulte les statistiques le lundi matin. Les lectures qui le justifient sont à mettre en cache dans Redis. Sur les statistiques, un retard de quelques minutes est acceptable.

Mesures demandées : 20 clients simultanés, 20 requêtes chacun, avec et sans cache, sur le planning et sur une statistique. Indicateurs : moyenne, écart-type, p95, nombre d'échecs.

4. Rapport de fréquentation. Les tourniquets enregistrent chaque entrée et chaque sortie ; l'export des deux dernières saisons est joint. Le rapport de fin de saison est aujourd'hui produit à la main, avec un notebook qui tourne plus d'une minute. Il prend la forme d'un PDF avec graphiques ; son contenu est détaillé en pièce jointe 2.

L'hébergeur coupe toute requête HTTP au-delà de 30 secondes : le rapport ne peut pas être calculé pendant la requête. La demande est déposée dans une file Redis, un worker Python (Celery) produit le PDF, et l'avancement se suit jusqu'au téléchargement.

5. Livraison. L'ensemble démarre avec `docker compose up`, sur un poste où seul Docker est installé. Un dossier par service, chacun avec son Dockerfile. Une collection Postman couvre toutes les routes, suivi du rapport compris.

Hélène Cattin

Pièce jointe 1 · La base SQLite

Schéma laissé par le prestataire. Les flèches indiquent les clés étrangères.

Table	Lignes	Colonnes
clubs	5	id, code, nom, ville, adresse, capacite (personnes présentes en même temps, au plus), horaires (texte JSON), ouvert_le
formules	5	id, code, libelle, prix_mensuel_centimes (le tarif actuel), engagement_mois, cours_collectifs (0 ou 1), tous_les_clubs (0 ou 1)
adherents	11 115	id, badge, prenom, nom, email, telephone, date_naissance, club_id → clubs (club de rattachement), inscrit_le
abonnements	13 248	id, adherent_id → adherents, formule_id → formules, debut, fin (vide tant que le contrat court), prix_mensuel_centimes (le prix signé), statut : actif, suspendu, termine ou resilie
coachs	34	id, prenom, nom, email, club_id → clubs, specialites (saisie libre), embauche_le, actif (0 ou 1)
activites	13	id, nom, categorie, intensite (1 à 3), duree_min
seances	17 398	id, activite_id → activites, coach_id → coachs, club_id → clubs, salle, debut (AAAA-MM-JJ HH:MM, heure de Paris), places, annulee (0 ou 1)
reservations	200 308	id, seance_id → seances, adherent_id → adherents, reservee_le, statut : confirmee, annulee, presente ou absente. Une seule réservation par adhérent et par séance.

Remarques.

- Les prix sont en centimes : 4690 correspond à 46,90 €.
- Les séances vont du 2 septembre 2024 à fin février 2027 : le planning est publié à l'avance. Les réservations des séances futures sont **confirmee** ; celles des séances passées sont **presente**, **absente** ou **annulee**.
- Un contrat en cours a une **fin** vide. Le statut **suspendu** correspond à un adhérent blessé : il ne vient plus, mais son contrat continue.
- Consultation : `sqlite3 data/forme-comtoise.sqlite` (commandes `.tables` et `.schema`), ou DB Browser for SQLite.

Pièce jointe 2 · L'export des tourniquets

`passages-2024-2026.csv` : 1 133 982 lignes plus l'en-tête, du 1^{er} septembre 2024 au 31 août 2026. Séparateur point-virgule, UTF-8, trié par club puis par heure.

```
horodatage;club;badge;sens
2024-09-01 09:03:34;BES-CENTRE;B057447;E
2024-09-01 09:05:52;BES-CENTRE;B090598;E
2024-09-01 09:07:15;BES-CENTRE;B070626;E
```

Colonne	Contenu
horodatage	heure de Paris, à la seconde
club	le code du club, celui de la table <code>clubs</code>
badge	le badge de l'adhérent, celui de la colonne <code>badge</code> de <code>adherents</code>
sens	E pour une entrée, S pour une sortie

Défauts connus de l'export. Un même passage peut être enregistré deux fois, à quelques secondes d'écart. Des sorties manquent. Quelques sorties n'ont pas d'entrée. Le badge B000000 est celui du technicien du prestataire, qui teste les tourniquets avant l'ouverture.

Contenu attendu du rapport

Pour une période (par défaut la dernière saison, du 1^{er} septembre 2025 au 31 août 2026), comparée à la même période un an plus tôt :

1. **Une synthèse par club** : le nombre de visites, le nombre de visiteurs différents, la durée médiane d'une visite, le pic de présence (le plus grand nombre de personnes présentes en même temps) et le jour où il a eu lieu.
2. **Pour chaque club et chaque mois** : l'occupation par jour de la semaine et par heure ; le pic de chaque journée comparé à la capacité ; la durée des visites ; l'écart avec le même mois de la saison précédente.
3. **Les journées à risque** : celles où le pic a atteint 90 % de la capacité, et celles où il l'a dépassée.
4. **Les cartes prêtées** : le même badge qui entre dans un second club moins de 40 minutes après être entré dans un premier.
5. **Les adhérents inactifs** : contrat actif le dernier jour de la période, et aucune visite pendant les 30 derniers jours. Club par club.

Définition : une visite est une entrée suivie de sa sortie, pour le même badge, dans le même club, le même jour. Le traitement des entrées sans sortie est libre ; il est décrit dans le rapport.

Consignes du projet

Travail individuel. La date limite est celle du devoir Moodle. L'usage d'un agent de code est autorisé ; l'étudiant doit pouvoir expliquer l'ensemble du code rendu.

Rendu

Un seul fichier zip, nommé `nom-prenom.zip`, avec la structure suivante :

```
nom-prenom/
  docker-compose.yml      toute la pile, MongoDB et Redis compris
  README.md
  migration/              Dockerfile, et le script SQLite vers MongoDB
  api/                    Dockerfile, et l'API Node.js
  worker/                 Dockerfile, et le worker Python
  bench/                  le script de charge et les mesures brutes (annexe A)
  postman/                la collection exportée, format v2.1 (annexe B)
  data/                   vide : on y copie les deux fichiers de l'archive
```

Exclus du zip : `node_modules`, `__pycache__`, les deux fichiers de données, les PDF générés, le dossier `.git`.

Contenu du README

1. **Lancement** : les commandes, dans l'ordre, sur un poste où seul Docker est installé.
2. **Décisions de modélisation** : une ligne par table SQLite, avec ce qu'elle devient (collection, document imbriqué, copie, référence), la raison, et la remontée des clubs, de (a) à (i), qui la justifie. Puis un exemple de document par collection.
3. **Routes** : méthode, chemin, rôle, codes de réponse possibles.
4. **Mesures** : un tableau avec et sans cache, pour le planning et pour une statistique (moyenne, écart-type, p95, nombre d'échecs, débit), et leur interprétation. Voir l'annexe A.
5. **Rapport** : durée de génération mesurée, et règles de calcul (entrées sans sortie, doubles passages, badge de test).
6. **Limites connues**.

Contraintes techniques

- API en Node.js 22 ou plus, avec Express et Mongoose. Worker en Python, avec Celery. Redis sert à la fois de cache et de file d'attente.
- MongoDB fait partie de la pile Compose, avec l'image `mongo:8.2`. Les images `mongo:8.0` et `mongo:latest` ne démarrent pas avec les versions récentes de Docker Desktop (noyau Linux 6.19 et plus).
- Les fichiers de données sont montés en lecture seule depuis `./data`, jamais copiés dans une image.
- Le cache se désactive par une variable d'environnement, sans modification du code.
- L'API utilise au moins trois pipelines d'agrégation, dont au moins deux avec `$lookup`.
- La migration se relance sans erreur et sans créer de doublons.

Barème

Partie	Points	Critères
Modélisation et migration	6	les décisions et leurs raisons, reliées aux remontées des clubs ; cinq collections au plus ; types convertis et données nettoyées ; migration rejouable
API	5	routes REST (verbes, noms, codes de réponse) ; agrégations et <code>\$lookup</code> justes ; règles de réservation, y compris pour deux réservations simultanées
Cache et mesures	4	ce qui est mis en cache et pourquoi ; clés et durées de vie ; protocole 20×20 avec et sans cache ; lecture des chiffres
Rapport en arrière-plan	3	réponse 202 puis suivi ; worker Celery ; contenu du PDF ; un échec visible dans le statut
Livraison	2	démarrage par <code>docker compose up</code> ; README ; collection Postman, suivi du rapport compris

Annexe A · Le dossier bench : le test de charge

Un test de charge envoie à l'API un grand nombre de requêtes simultanées et mesure le temps de réponse de chacune. Il mesure le service rendu quand plusieurs utilisateurs sont connectés en même temps.

Protocole

- **20 clients simultanés, 20 requêtes chacun** : 20 boucles lancées en parallèle ; chaque boucle envoie ses 20 requêtes l'une après l'autre, en attendant chaque réponse. Soit 400 requêtes par campagne.
- **Échauffement** : une vingtaine de requêtes envoyées avant la mesure et non comptées. Elles ouvrent les connexions et, cache activé, remplissent Redis.
- **Deux campagnes par route**, cache désactivé puis activé. Entre les deux, seule la variable d'environnement du cache change : le service `api` est redémarré, puis Redis est vidé avec `docker compose exec redis redis-cli FLUSHALL`.
- **Deux routes** : le planning d'un club pour une semaine, et une statistique de la direction sur une longue période.

Indicateurs

Calculés en millisecondes, sur les requêtes réussies (réponse 200).

Indicateur	Définition
moyenne	somme des temps divisée par le nombre n de requêtes réussies
écart-type	dispersion des temps x_i autour de la moyenne m : $s = \sqrt{\frac{1}{n-1} \sum_i (x_i - m)^2}$
p95	temps sous lequel se trouvent 95 % des requêtes : temps triés par ordre croissant, valeur de rang $\lceil 0,95 \times n \rceil$
échecs	requêtes sans réponse 200 (erreur, ou connexion refusée) ; leurs temps n'entrent dans aucun calcul, leur nombre est donné à part
débit	nombre de requêtes divisé par la durée totale de la campagne, en requêtes par seconde

Principe du script

Node.js 22 ou plus, sans dépendance. L'extension `.mjs` autorise le `await` au niveau du fichier.

```
// bench/charge.mjs      node bench/charge.mjs http://localhost:3000/...
const url = process.argv[2];
const CLIENTS = 20;
const REQUETES = 20;
const mesures = [];

async function client(numero) {
  for (let rang = 1; rang <= REQUETES; rang++) {
    const debut = performance.now();
    let statut = 0;           // 0 : pas de reponse
    try {
      const reponse = await fetch(url);
      await reponse.arrayBuffer(); // lire le corps : on mesure la reponse entiere
      statut = reponse.status;
    } catch {}               // connexion refusee ou coupee
    mesures.push({ client: numero, rang, ms: performance.now() - debut, statut });
  }
}

const t0 = performance.now();
await Promise.all(Array.from({ length: CLIENTS }, (_, i) => client(i + 1)));
const duree = (performance.now() - t0) / 1000; // secondes

// A ecrire : l'echauffement, les cinq indicateurs, et l'enregistrement des mesures brutes.
```

D'autres outils (k6, Apache Bench, autocannon) sont acceptés s'ils suivent le même protocole et donnent les cinq indicateurs.

Contenu du dossier et du README

```
bench/
  charge.mjs
  resultats/
    planning-sans-cache.csv      une ligne par requete : client, rang, ms, statut
    planning-avec-cache.csv
    statistique-sans-cache.csv
    statistique-avec-cache.csv
```

Dans le README, un tableau de quatre lignes, puis l'interprétation :

Route	Cache	Moyenne	Écart-type	p95	Échecs	Débit
planning	sans	... ms	... ms	... ms	... / 400	... req/s
planning	avec					
statistique	sans					
statistique	avec					

Annexe B · Le dossier postman : la collection de requêtes

Postman est une application gratuite qui envoie des requêtes HTTP et affiche les réponses. L'API n'ayant pas d'interface, chaque route est appelée et vérifiée depuis Postman.

Notions

Notion	Rôle
Collection	ensemble de requêtes enregistrées, rangées en dossiers, rejouables dans l'ordre
Variable	valeur réutilisée dans les requêtes. <code>{{baseUrl}}</code> vaut <code>http://localhost:3000</code> ; elle se définit dans l'onglet <i>Variables</i> de la collection
Script	code JavaScript exécuté après la réponse : onglet <i>Scripts</i> , partie <i>Post-response</i> (onglet <i>Tests</i> dans les anciennes versions)
Runner	exécute toute la collection dans l'ordre : bouton <i>Run</i> de la collection
Export	menu ... de la collection, puis <i>Export</i> , format <i>Collection v2.1</i> : un fichier JSON, placé dans <code>postman/</code>

Vérifier une réponse, garder une valeur

```
// apres POST /rapports : verifier le code, garder l'identifiant
pm.test("202", () => pm.response.to.have.status(202));
pm.collectionVariables.set("rapport", pm.response.json().id);
```

Suivre le rapport jusqu'à la fin

La requête *Suivre le rapport* se rejoue toutes les 2 secondes tant que le rapport n'est pas terminé. Les noms de champs et de statuts sont ceux de l'API rendue.

```
// requete "Suivre le rapport" : GET {{baseUrl}}/rapports/{{rapport}}
const r = pm.response.json();
if (r.statut === "en_attente" || r.statut === "en_cours") {
  pm.execution.setNextRequest("Suivre le rapport"); // rejouer cette meme requete
  setTimeout(() => {}, 2000); // apres une pause de 2 s
} else {
  pm.test("rapport termine", () => pm.expect(r.statut).to.eql("termine"));
}
```

`pm.execution.setNextRequest` n'agit que dans le Runner : envoyée seule, la requête ne boucle pas.

Contenu attendu de la collection

- une requête par route de l'API, rangées en dossiers;
- sur chaque requête, au moins un test du code de réponse;
- la séquence du rapport : demande, suivi, téléchargement du PDF.

La collection exportée s'exécute aussi en ligne de commande, comme dans le Runner : `npx newman run postman/forme-comtoise.postman_collection.json`.

Cas pédagogique fictif : l'entreprise, les personnes et les données sont inventées pour ce projet.